

SMOLR / Demo Embedded Disassembler Design

Status

Draft design document.

This document captures the design for a small RISC-V disassembler that serves two related purposes:

1. A development tool for SMOLR and demoscene binaries.
2. An embeddable runtime effect inside a demo, able to disassemble the demo's own code into text for graphical use.

The design principle is that the full disassembler may be broad and useful during development, while the embedded version should be automatically reduced to only the instruction families and formatting features needed by a particular demo.

1. Core Idea

The disassembler should be designed as a library of small atoms.

A complete build can become a useful external disassembler for SMOLR-generated executables. A demo build can link only the subset needed for that specific executable, using the same discipline as the rest of the demo system: softsynth atoms, graphics atoms, SPINE atoms, instrument tables, effect tables, and decode atoms should all be included only when reachable.

The embedded disassembler is therefore not a gimmick. It is both:

- a debugging and inspection tool for size-constrained RISC-V binaries;
- an artistic source generator, producing real code-derived text that can be used as visual material.

One intended visual use is a cave scene: a procedurally generated cave system where firelight flickers across walls covered in ancient carvings. The carvings are not arbitrary text. They are the demo disassembling itself.

2. Goals

2.1 Development Tool Goals

The development version should help inspect SMOLR-generated executables and answer questions such as:

- Which instruction families are present after final linking?
- Which compressed instructions appear?
- Which instructions are not yet covered by the embedded decoder?

- Did linker relaxation or instruction compression change the final code shape?
- Which decode atoms would be needed to disassemble this specific binary?
- Which parts of a library are being pulled in unexpectedly?

The development tool may support ELF parsing, symbols, section names, executable segment detection, and instruction coverage reporting.

2.2 Embedded Demo Goals

The embedded version should:

- walk a known executable memory range, usually `.text`;
- decode 16-bit and 32-bit RISC-V instructions;
- emit disassembly text into a buffer;
- use simple textual formatting suitable for later graphics processing;
- degrade gracefully when it sees an unknown instruction;
- be link-pruned to only the instruction families and emitters needed by the demo;
- avoid bulky features such as symbols, labels, ELF parsing, ABI register names, and pseudo-instruction prettification unless explicitly needed.

The embedded version is allowed to be incomplete if the fallback output remains useful.

3. Non-Goals

The embedded version does not initially aim to be a full replacement for `objdump`.

It does not need, by default:

- symbolic labels;
- relocation display;
- source-line mapping;
- ABI register names;
- pseudo-instruction reconstruction;
- complete CSR name tables;
- full RVA23 coverage;
- debug information parsing;
- exact reconstruction of original assembly source.

The goal is to disassemble the actual machine code that exists in the final linked executable, not to recover what the programmer originally wrote.

4. Two Build Personalities

4.1 Full Development Disassembler

This build may include:

- ELF parser;
- executable segment or `.text` section discovery;
- optional symbol table support;
- optional relocation display;
- instruction coverage reporting;
- full or broad RISC-V decode coverage;
- optional pretty-printing modes;
- unknown-instruction reporting.

This version is allowed to be larger.

4.2 Embedded Runtime Disassembler

This build should include only:

- memory range walker;
- instruction length detection;
- minimal output buffer writer;
- selected instruction family decoders;
- simple register and immediate formatting;
- fallback `.2byte` / `.4byte` output.

This version should be aggressively size-pruned.

5. Architectural Principle: Atoms and Reachability

The disassembler should be split into independently removable atoms.

An atom can be:

- one function;
- one small family decoder;
- one string table;
- one immediate formatter;
- one register-name emitter;
- one optional output style;
- one instruction extension capsule.

The final executable should include only atoms reachable from the chosen root symbols.

Conceptually:

```
roots = [_start, demo_main, selected_effects]
reachable = graph walk over symbol references and relocations
emit only reachable atoms
layout / relax / compress
```

The same approach applies to the whole demo:

```
demo entry
-> SPINE event system
    -> cave text-carving effect
        -> embedded disassembler
            -> RV64I subset decoder
            -> RVC subset decoder
            -> fallback emitter
```

Everything else is discarded.

6. Granularity Rules

Maximum granularity is useful only when it does not increase total size.

6.1 Split Aggressively When

Split into separate atoms for:

- optional instruction extensions;
- rare instruction families;
- large decode functions;
- optional output formats;
- optional symbol handling;
- optional ELF handling;
- large lookup tables;
- feature-specific strings;
- development-only helpers.

6.2 Group When Smaller

Group atoms when splitting would cause bloat from:

- excessive alignment padding;
- too many relocations;
- too many long calls or jumps;
- tiny string fragments;
- repeated prologue/epilogue code;

- tables that compress better when contiguous;
- helpers that are always used together.

The practical rule:

One atom per independently optional thing.

Not:

One atom per microscopic fragment.

7. Proposed Module Layout

7.1 Core Runtime Modules

```
disasm_core/  
  walk_range  
  decode_instruction_length  
  output_buffer_writer  
  emit_char  
  emit_string  
  emit_space  
  emit_newline  
  emit_hex_u16  
  emit_hex_u32  
  emit_signed_decimal_or_hex  
  emit_xreg_numeric  
  emit_freg_numeric  
  emit_vreg_numeric  
  fallback_dot_halfword  
  fallback_dot_word
```

The embedded version should always start here.

7.2 RV64I Decode Modules

```
disasm_rv64i/  
  decode_lui_auipc  
  decode_jal_jalr  
  decode_branch  
  decode_load  
  decode_store  
  decode_op_imm  
  decode_op  
  decode_system_minimal
```

7.3 Compressed Decode Modules

```
disasm_rvc/  
  decode_c_quadrant_0  
  decode_c_quadrant_1  
  decode_c_quadrant_2  
  decode_c_addi  
  decode_c_li  
  decode_c_lui  
  decode_c_mv_add  
  decode_c_ld_sd  
  decode_c_lw_sw  
  decode_c_ldsp_sdsp  
  decode_c_lwsp_swsp  
  decode_c_j_jal  
  decode_c_jr_jalr  
  decode_c_beqz_bnez  
  decode_c_shift_andi
```

Compressed instruction support is essential for size-constrained RISC-V demos.

7.4 Optional Extension Capsules

```
disasm_ext_m/  
  decode_muldiv  
  
disasm_ext_a/  
  decode_atomic  
  
disasm_ext_f/  
  decode_float_single  
  
disasm_ext_d/  
  decode_float_double  
  
disasm_ext_b/  
  decode_bitmanip_subset  
  
disasm_ext_v/  
  decode_vector_subset  
  
disasm_ext_zfa/  
  decode_zfa_subset  
  
disasm_ext_zicond/  
  decode_conditional_zero  
  
disasm_ext_csr/
```

```
decode_csr_numeric
csr_name_table_optional
```

Each extension should be optional.

7.5 Development-Only Modules

```
disasm_elf/
  parse_elf_header
  parse_program_headers
  find_executable_segments
  find_text_section_optional
  parse_symbol_table_optional
  parse_string_table_optional

disasm_report/
  instruction_coverage_report
  unsupported_instruction_report
  required_decoder_atom_report
  size_contribution_report

disasm_pretty/
  abi_register_names
  pseudo_instruction_printer
  symbolic_branch_labels
  symbol_plus_offset_formatting
```

These should not be pulled into the embedded demo build unless intentionally requested.

8. Embedded Runtime Flow

For the embedded version, avoid ELF parsing when possible. The demo can expose linker symbols for known code ranges:

```
__text_start
__text_end
```

Then runtime disassembly is simply:

```
p = __text_start
pc = runtime or link-time address of __text_start
out = text_buffer

while p < __text_end:
    h = load16(p)
```

```

if (h & 3) != 3:
    decode16(h, pc, out)
    p += 2
    pc += 2
else:
    w = load32(p)
    decode32(w, pc, out)
    p += 4
    pc += 4

emit_newline(out)

```

The decoder should never fail catastrophically. Unknown instructions become data directives.

```

.2byte 0x1141
.4byte 0x00000517

```

This is both useful and visually acceptable.

9. Development Tool Flow

The external development tool may operate on an ELF file.

Possible flow:

```

open ELF file
validate ELF64 little-endian RISC-V
read ELF header
read program headers
for each executable PT_LOAD segment:
    disassemble bytes from p_offset to p_offset + p_filesz
    use p_vaddr as pc base
collect instruction coverage
report unknown / unsupported instructions
report required decoder atoms

```

Section-header support can be added later. Program headers are enough for a first executable-image view.

10. Output Style

10.1 Minimal Embedded Style

Preferred first embedded style:

```
1348 c.addi x2,-16
134a c.sdsp x1,8(x2)
134c c.mv x15,x10
134e lui x10,0
1352 addi x10,x10,84
```

Properties:

- numeric register names;
- no ABI names;
- no labels;
- no pseudoinstructions;
- compact spacing;
- raw branch targets or immediates;
- unknowns as `.2byte` / `.4byte`.

10.2 Development Style

Possible external style:

```
00001348: 1141      c.addi    x2,-16
0000134a: e406      c.sdsp    x1,8(x2)
0000134c: 87aa      c.mv      x15,x10
0000134e: 00000537  lui       x10,0x0
```

Development style may include raw bytes, addresses, symbols, and comments.

10.3 Artistic Style Layer

The disassembler should output plain text. The graphical layer can then distort, arrange, damage, or stylize it.

Possible cave-wall transformations:

- irregular spacing;
- broken lines;
- faded characters;
- text following wall contours;
- glowing firelight modulation;
- partial erosion of instruction text;
- mixing decoded instructions with fallback `.word` fragments;

- carving depth based on opcode class or instruction frequency.

This keeps the disassembler itself clean and reusable.

11. Dispatch Strategy

Avoid a single giant dispatch table that references every decoder. Such a table can accidentally keep the entire decoder library alive.

Risky pattern:

```
decode_table:  
    pointer decode_addi  
    pointer decode_slti  
    pointer decode_xori  
    pointer decode_vector_op  
    pointer decode_atomic
```

If the table is referenced, every relocation in the table may keep every listed decoder.

Preferred embedded pattern:

```
decode32:  
    inspect opcode  
    branch only to enabled family decoders  
    otherwise fallback .4byte
```

Feature capsules should be referenced only when selected.

For the full development tool, a denser table-driven approach may be acceptable, because total size is less critical.

12. Coverage-Driven Decoder Growth

The decoder should grow based on actual binaries, not theoretical full ISA coverage.

A development tool should scan the final linked executable and produce a report like:

```
Instruction families present:
```

```
RV64I:  
    addi      83  
    lui       12
```

```

jal      21
beq      18

RVC:
c.addi   141
c.sdsp   38
c.ldsp   37

M:
mul      4

Unsupported:
opcode 0x5b / funct3 0x2: 2 occurrences

```

This report should guide which embedded decode atoms are needed.

Eventually SMOLR could emit a required-disassembler-feature set directly.

13. Interaction with SMOLR

SMOLR should ideally understand code/data atoms, symbol dependencies, relocations, alignment requirements, and optional feature roots.

A possible internal atom model:

```

atom:
  kind: code | data | rodata | metadata
  bytes
  exported symbols
  imported symbols
  relocation records
  alignment
  flags
  compression hints

```

The disassembler can help SMOLR development in two ways:

1. External inspection of SMOLR output.
2. Internal feature selection for embedded self-disassembly.

Possible SMOLR report:

```

Required embedded disassembler atoms:
disasm_core
disasm_rv64i.decode_op_imm
disasm_rv64i.decode_load
disasm_rv64i.decode_store

```

```
disasm_rvc.decode_c_addi
disasm_rvc.decode_c_ldsp_sdsp
disasm_ext_m.decode_muldiv
```

14. Linker / Section Strategy for Assembly

For handwritten assembly, use section-per-function or section-per-feature where useful.

Example macro style:

```
.macro FUNC name
    .section .text.\name, "ax", @progbits
    .globl \name
    .type \name, @function
\name:
.endm

.macro ENDFUNC name
    .size \name, .-\name
.endm
```

Example function:

```
FUNC disasm_emit_xreg
    # emit x0..x31 using numeric formatting
    ret
ENDFUNC disasm_emit_xreg
```

Feature-specific strings can be placed in separate rodata sections:

```
.section .rodata.disasm.str_c_addi, "a", @progbits
str_c_addi:
    .ascii "c.addi\0"
```

But very small strings may be better grouped by family:

```
.rodata.disasm.rv64i_strings
.rodata.disasm.rvc_strings
.rodata.disasm.m_strings
```

15. Code/Data Separation and Ranges

The embedded disassembler should preferably disassemble known executable ranges rather than blindly scanning the entire loaded image.

Reason: a loaded executable may contain:

- code;
- constants;
- jump tables;
- literal data;
- string data;
- packed resources;
- alignment padding.

Disassembling all bytes can produce nonsense. That may be artistically useful later, but the first version should walk clean code ranges.

Recommended first target:

```
.text only
```

Possible later target:

```
.text plus selected rodata fragments, intentionally rendered as mysterious  
false-code carvings
```

16. Instruction Length Detection

RISC-V instruction length detection is the first essential primitive.

For the common 16-bit/32-bit case:

```
load 16 bits  
if low two bits are not 11:  
    compressed 16-bit instruction  
else:  
    32-bit instruction for current target subset
```

The first embedded version can treat only 16-bit and 32-bit instructions. Longer encodings are not needed for the initial SMOLR/demo target unless deliberately introduced.

17. Unknown Instruction Policy

Unknown instructions must not stop the disassembler.

Policy:

- unknown 16-bit instruction: emit `.2byte 0xhhhh` ;
- unknown 32-bit instruction: emit `.4byte 0xwwwwwwww` ;
- continue scanning at the next instruction boundary.

This makes the embedded version robust even when decoder coverage is incomplete.

18. Size Targets

Approximate size targets for handwritten RV64 assembly:

Fallback-only walker:

< 1 KiB to 2 KiB

Minimal useful embedded disassembler:

2 KiB to 5 KiB

Practical embedded disassembler for real demo subset:

4 KiB to 8 KiB

Broader embedded disassembler with several extensions:

8 KiB to 16 KiB

Full development disassembler:

size less critical; likely tens of KiB or more

These are planning targets, not guarantees.

Actual size depends heavily on:

- output formatting richness;
 - number of instruction families;
 - string storage;
 - register name strategy;
 - use of tables;
 - alignment;
 - compressed instruction density;
 - linker relaxation;
 - SMOLR's atom pruning effectiveness.
-

19. Minimal Viable Milestone

The smallest useful first milestone is not a real mnemonic decoder.

It is:

```
code memory -> fallback disassembly text buffer -> graphics/SPINE input
```

Implement only:

```
disasm_walk_range
disasm_decode_instruction_length
disasm_emit_address_optional
disasm_emit_dot_halfword
disasm_emit_dot_word
disasm_emit_newline
```

Output example:

```
00001234 .2byte 0x1141
00001236 .2byte 0xe406
00001238 .4byte 0x00000517
```

This proves:

- memory walking works;
- instruction boundary detection works;
- output buffer handling works;
- text can be fed into graphics;
- the effect pipeline exists.

Then real instruction families can be added one at a time.

20. Suggested Milestones

Milestone 1: Fallback Walker

- Walk a known memory range.
- Detect 16-bit vs 32-bit instructions.
- Emit `.2byte` / `.4byte` lines.
- Feed output into a text buffer.

Milestone 2: Minimal RV64I Decode

Add common 32-bit instructions:

- `lui`
- `auipc`
- `jal`
- `jalr`
- branches
- loads
- stores
- `addi`
- basic integer register-register ops

Milestone 3: Essential RVC Decode

Add common compressed instructions:

- `c.addi`
- `c.li`
- `c.lui`
- `c.mv`
- `c.add`
- `c.ldsp`
- `c.sdsp`
- `c.lwsp`
- `c.swsp`
- `c.ld`
- `c.sd`
- `c.lw`
- `c.sw`
- `c.j`
- `c.jr`
- `c.jalr`
- `c.beqz`
- `c.bnez`

Milestone 4: Coverage Reporter

Create external tool mode that scans a binary and reports:

- decoded instructions;
- unknown instructions;
- instruction family counts;
- required decoder atoms.

Milestone 5: SMOLR Integration

Use the coverage report or SMOLR's own internal knowledge to select embedded disassembler atoms.

Milestone 6: Cave-Wall Integration

Use the output buffer as graphics input:

- texture source;
- carving mask;
- wall inscription stream;
- glyph-depth source;
- firelight modulation input.

Milestone 7: Optional Extension Growth

Add only the extensions actually present in final binaries:

- M;
- A;
- B subset;
- F/D subset;
- V subset;
- CSR numeric decode;
- Zfa/Zicond/etc. only if needed.

21. Open Design Questions

1. Should the embedded disassembler print addresses, or only instruction text?
2. Should immediates be printed in decimal, hex, or both?
3. Should branch targets be absolute addresses, relative offsets, or omitted?
4. Should the cave-wall effect prefer compact text or visually varied text?
5. Should the embedded version support only `.text`, or intentionally include selected data ranges?
6. How should SMOLR describe the instruction families it emitted?
7. Should instruction-family selection be automatic, manual, or both?
8. What is the smallest register formatter that still looks good?
9. Is it worth including ABI register names in the development tool only?
10. Should unknown instruction output preserve raw bytes in little-endian order or normalized word order?

22. Design Doctrine

The disassembler should follow the same discipline as the rest of the demo system:

Build a large library of tiny atoms. Compose only the atoms needed for the piece.

For the embedded demo version:

Correct enough, tiny, graceful fallback, visually useful.

For the development version:

Broad enough to inspect SMOLR output and guide size reduction.

The full disassembler is a tool. The embedded disassembler is both a tool and an instrument.

In the cave scene, the executable becomes archaeology: the machine reads its own bones and writes them on stone.